Project - High Level Design
On
CI/CD pipeline for Node.js
Retail Application

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Kratin Verma	EN22CS301517
2.	Karishma Patidar	EN22CS301484
3.	Kanishka Patidar	EN22CS301480
4.	Jay Yadav	EN22CS301462

Group Name: Group 01D5

Project Number: DO-01

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 4th

1. Introduction.

This document describes the High-Level Design of the CI/CD Pipeline for the Node.js Retail Application. It provides an overview of system architecture, components, workflows, and deployment strategy using DevOps tools.

1.1 Scope of the document.

This document covers:

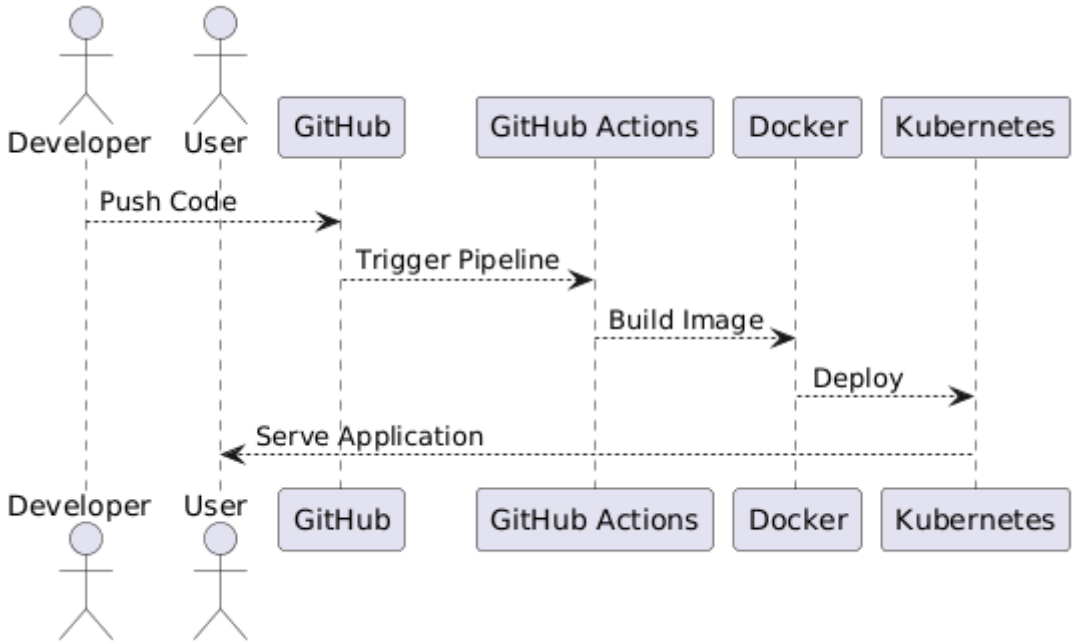
- System architecture
- CI/CD workflow
- Application deployment
- Cloud infrastructure (AWS EC2)
- Containerization using Docker
- Orchestration using Kubernetes

1.2 Intended Audience

- Developers
- DevOps Engineers
- Project Evaluators
- Faculty Members

1.3 System overview.

The system automates the software development lifecycle using CI/CD. The application is containerized and deployed on cloud infrastructure.



2. System Design.

<u>The system follows a layered architecture:</u>

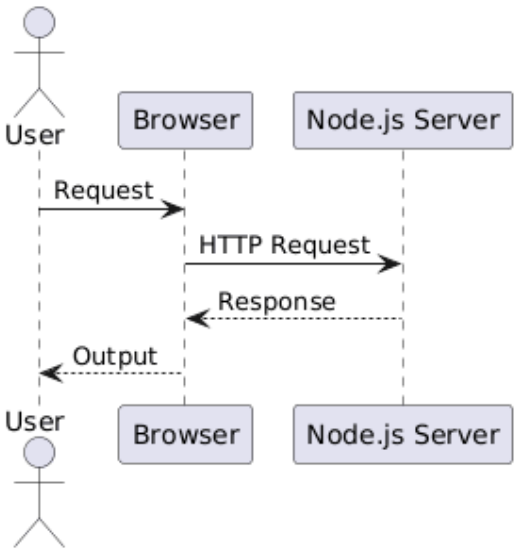
- Presentation Layer (User Interface)
- Application Layer (Node.js App)
- Container Layer (Docker)
- Orchestration Layer (Kubernetes)
- Infrastructure Layer (AWS EC2)

2.1 Application Design

The application is built using Node.js and Express.js.

<u>Modules:</u>

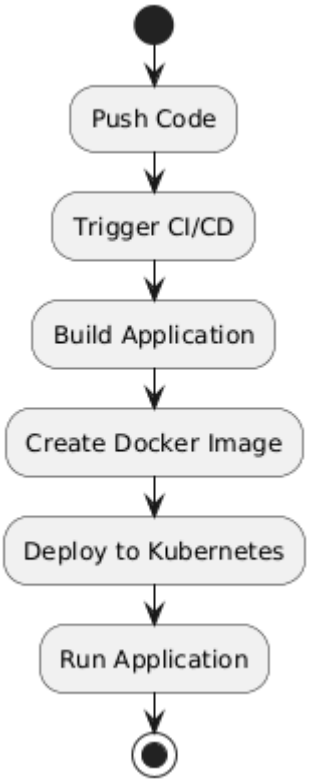
- Server Module
- Routing Module
- API Module



2.2 Process Flow.

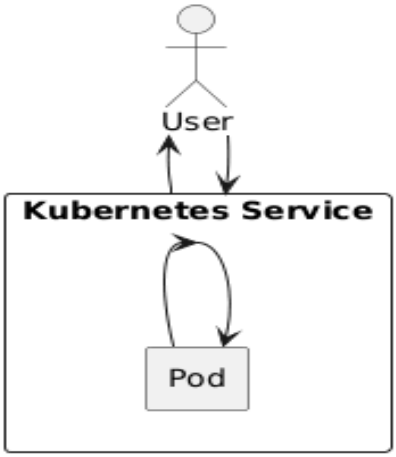
Steps:

1) Developer pushes code
2) CI/CD pipeline triggers
3) Application builds
4) Docker image is created
5) Image is deployed to Kubernetes
6) Application is accessible to users



2.3 Information Flow.

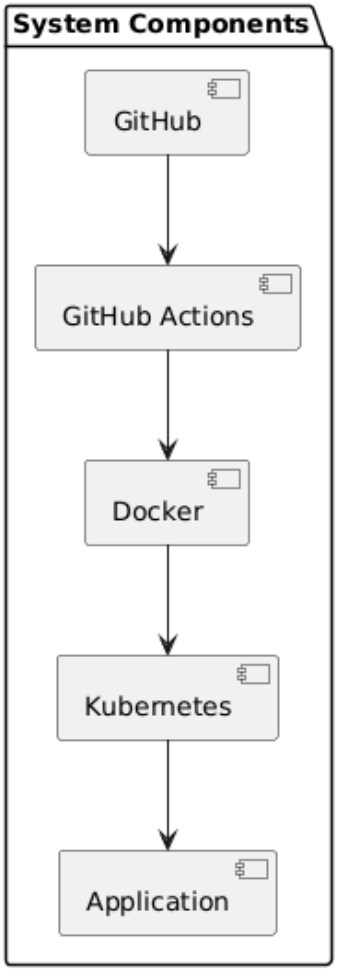
User requests are processed through Kubernetes services and pods.



2.4 Components Design

Components:

- GitHub → Code repository
- GitHub Actions → CI/CD automation
- Docker → Containerization
- Kubernetes → Orchestration
- Amazon Web Services → Hosting



2.5 Key Design Considerations

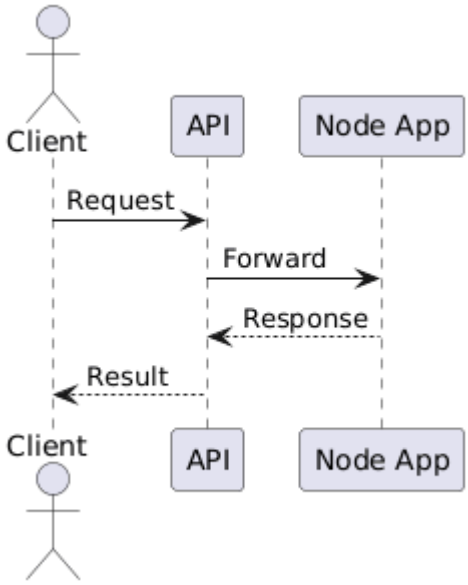
- Scalability using Kubernetes
- Automation using CI/CD
- High availability
- Security using AWS security groups
- Container-based deployment

2.6 API Catalogue.

Base URL: http://<EC2-IP>:PORT

APIs:

- GET / → Returns application status
- GET /health → Health check
- GET /products → Fetch products (optional)



3. Data Design.

The application can use simple JSON-based data or database storage for products and users.

3.1 Data Model

Basic data entities:

- User
- Product
- Order

3.2 Data Access Mechanism

Data is accessed using REST APIs implemented in Node.js.

3.3 Data Retention Policies

- Logs stored temporarily
- Application data stored as required
- No sensitive data stored permanently

3.4 Data Migration

Data migration is minimal and can be handled using scripts if database is used.

4. Interfaces

- REST APIs
- Web Browser Interface

5. State and Session Management

- Stateless architecture
- Each request handled independently

6. Caching

Basic caching can be implemented for performance improvement

7. Non-Functional Requirements

- Scalability
- Reliability
- Performance
- Security

7.1 Security Aspects

- AWS Security Groups
- Secure API endpoints
- Container isolation

7.2 Performance Aspects

- Fast deployment using containers
- Efficient resource utilization
- Load balancing via Kubernetes

8. References

- Node.js Documentation
- Docker Documentation
- Kubernetes Documentation
- GitHub Actions Docs
- AWS EC2 Documentation